

Création de Chaînes

On a déjà vu:

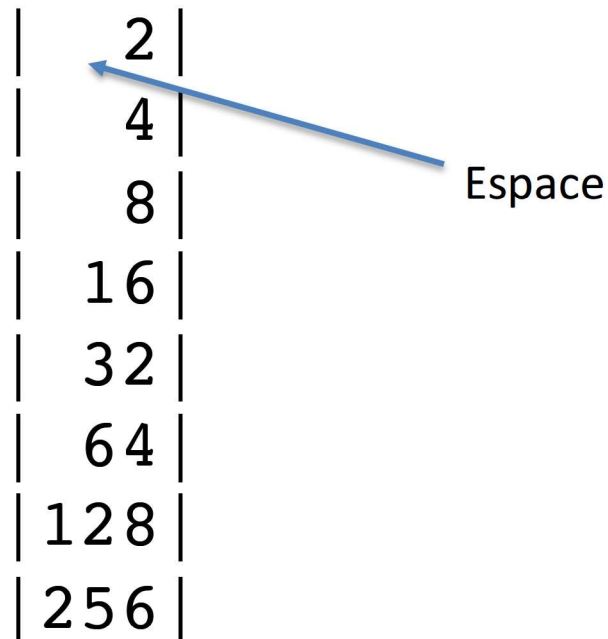
```
def hello_message(name):  
    message = "Hello, " + name  
    return message
```

```
>>> hello_message("Charles")  
'Hello, Charles'  
>>> hello_message(str(42))  
'Hello, 42'
```

Ici, on utilise **+** pour concaténer des chaînes

Création de Chaînes

- La concaténation peut donner du code qui est
 - Long
 - Difficile à lire
- Comment imprimer ce tableau?



2
4
8
16
32
64
128
256

Création de Chaînes

- En utilisant ce qu'on a vu jusque maintenant:

```
for k in range(1, 9):  
    number = str(2**k)  
    nrspaces = 3 - len(number)  
    print ( "|" + " " * nrspaces + number + "|" )
```

	2	
	4	
	8	
	16	
	32	
	64	
	128	
	256	

- Plus simple:

```
for k in range(1, 9):  
    print ( "|{0:>3}|" .format ( 2**k ) )
```

Création de Chaînes

```
print ( "{0:>3}".format( 2**k ) )
```

Chaîne

Méthode appliquée
sur chaîne

Argument

- La méthode "format" retourne une nouvelle chaîne basée sur "{0:>3}" et 2**k
- Comment comprendre "{0:>3}"?

Création de Chaînes

Les accolades définissent des champs de remplacement



```
>>> print ( "{1} X {0} X {1}".format ( 12, 34 ) )
34 X 12 X 34
                                0    1

>>> print ( "H: {0} - {1} - {0}".format ( 12, 34 ) )
H: 12 - 34 - 12
H: 34 - 12 - 12

>>> print ( "H: {0} - {{1}} - {0}".format ( 12, 34 ) )
H: 12 - {1} - 12
```

Création de Chaînes

Des indicateurs supplémentaires influencent le remplacement

Alignement (droit) Largeur (9)



```
>>> print ( "{0:>9}".format ( 12 ) )
```

```
_____12
7 espaces
```

```
>>> print ( "{0:^9}".format ( 1 ) )
```

```
____1____
4       4
```

```
>>> print ( "{0:^9}".format ( 123 ) )
```

```
____123____
3         3
```

Création de Chaînes

```
for k in range(1, 9):  
    print ( "{0:>3}|".format ( 2**k ) )
```

```
| 2 |  
| 4 |  
| 8 |  
|16 |  
|32 |  
|64 |  
|128|  
|256|
```

Création de Chaînes

```
for k in range(1, 9):  
    print ( "|{0:<4}|" .format ( 2**k ) )
```

```
| 2 |  
| 4 |  
| 8 |  
| 16 |  
| 32 |  
| 64 |  
| 128 |  
| 256 |
```

Création de Chaînes

Beaucoup d'autres méthodes peuvent être utilisées pour créer des chaînes:

```
>>> print ( "hello".upper ( ) )
```

```
HELLO
```

```
>>> print ( "HELLO".lower ( ) )
```

```
hello
```

```
>>> print ( "          text".strip ( ) )
```

```
text
```



Pas d'espaces

<https://docs.python.org/3/library/stdtypes.html#string-methods>

Extraction d'Informations de Chaînes

Exemple: comment créer une fonction pour compter le nombre de 'A' dans une chaîne?

```
def count_a ( s ):
    cnt = 0
    for c in s:
        if c == "A":
            cnt += 1
    return cnt
```

Une chaîne est une séquence de caractères

c est une chaîne avec un caractère

```
>>> count_a ( "BANANA" )
```

```
3
```

```
>>> count_a ( "BaNANA" )
```

```
2
```

Extraction d'Informations de Chaînes

Exemple: comment vérifier si à la position *i* il y a un "A"?

```
s = "BANANA"
i = 1
if s[i] == "A":
    print ( True )
else:
    print ( False )
```



Index entre crochets

True ou False ?

Extraction d'Informations de Chaînes

`s = "LOUVAIN"`

L	O	U	V	A	I	N
0	1	2	3	4	5	6
-7	-6	-5	-4	-3	-2	-1

<code>s[2]</code>	"U"
<code>s[-1]</code>	"N"
<code>s[1:3]</code>	"OU"
<code>s[-4:-1]</code>	"VAI"
<code>s[2:]</code>	"UVAIN"
<code>s[:2]</code>	"LO"
<code>s[:]</code>	"LOUVAIN"

Séquences

- On a vu quelques séquences:

- Nombres

- `s = [ 0 , 1 , 2 , 3 ]`

- `s = range( 4 )`

- Caractères

- `s = "LOUVAIN"`

- Permet de code comme

```
for e in s:  
    print ( " |{0:>10}| ".format ( e ) )
```

Séquences

- On a vu quelques séquences:

- Nombres

`s = [0,1,2,3]`

```
for e in s:  
    print ( "|{0:>10}|" .format ( e ) )
```

```
|          0 |  
|          1 |  
|          2 |  
|          3 |
```

Séquences

- On a vu quelques séquences:

- Caractères

`s = "LOUVAIN"`

```
for e in s:  
    print ( "|{0:>10}|" .format ( e ) )
```

```
|          L |  
|          O |  
|          U |  
|          V |  
|          A |  
|          I |  
|          N |
```

Séquences

- Comment créer :

```
|   LOUVAIN |  
| BRUXELLES |  
|       MONS |  
|   TOURNAI |
```

- Solution: séquences d'autres éléments, par exemple, une séquence de chaînes

```
for e in [ "LOUVAIN", "BRUXELLES", "MONS", "TOURNAI " ]:  
    print ( " |{0:>10}| ".format ( e ) )
```